كيف يعمل Secure Messenger — شرح كامل خطوة بخطوة؟

المرحلة 0 — توليد المفاتيح (مرة واحدة فقط)

قبل أي شيء، النظام يولّد زوج مفاتيح RSA:

```
┌──────────────────────────────────────┐
│        RSA Key Generator             │
│                                      │
│  Public Key  ──→ يُشارَك مع الجميع   │
│  Private Key ──→ يبقى سري عند Bob    │
└──────────────────────────────────────┘
```

> شبيه بـ صندوق بريد: أي شخص يقدر يحط رسالة فيه (المفتاح العام)، لكن فقط صاحبه يفتحه (المفتاح الخاص)

المرحلة 1 — Alice تشفّر الرسالة (Sender)

عندما تضغط "Encrypt & Send" يحدث هذا بالترتيب:

الخطوة 1 — توليد مفتاح AES عشوائي

```
CSPRNG ──→ مفتاح AES-256 (32 بايت) عشوائي
           + Nonce/IV (12 بايت) عشوائي
```

> مفتاح جديد لكل رسالة — حتى لو نفس النص، التشفير مختلف كل مرة

الخطوة 2 — تشفير الرسالة بـ AES-GCM

```
"Hello Bob!" + مفتاح AES + Nonce
        ↓
   AES-256-GCM
        ↓
 xK9#@mZ7!p... (Ciphertext)
 + Authentication Tag (يتأكد من السلامة تلقائياً)
```

الخطوة 3 — تغليف مفتاح AES بـ RSA

```
مفتاح AES (32 بايت)
        ↓
 RSA-OAEP + المفتاح العام لـ Bob
        ↓
 Encrypted AES Key (256 بايت)
```

> حتى لو اعترض أحد الحزمة — AES يقدر يفكّ مفتاح Bob الآن فقط

الخطوة 4 — حساب بصمة SHA-256

```
"Hello Bob!"
        ↓
   SHA-256
        ↓
 a3f8c2d1e9b4... (64 حرف ثابت)
```

> هذه البصمة ستُستخدم لاحقاً للتحقق أن الرسالة لم تُعدَّل

الحزمة النهائية المرسلة

```
{
  "ciphertext":      "xK9#@mZ7!p...",   ← الرسالة مشفّرة
  "iv":             "r4nDOm...",        ← الـ Nonce
  "encryptedAesKey": "RSA(AES_key)...", ← المفتاح مغلّف
  "msgHash":        "a3f8c2d1..."      ← البصمة
}
```

المرحلة 2 — الشبكة (القناة)

```
Alice ──────────────────────────────────► Bob
         يتنقل عبر الشبكة Packet

         إذا فعّلت MITM: ☠
Alice ──► [Attacker يعدّل الرسالة] ──► Bob
```

المرحلة 3 — Bob يفكّ ويتحقق (Receiver)

عندما تضغط "Decrypt & Verify":

الخطوة 1 — فك تغليف مفتاح AES

```
Encrypted AES Key
        ↓
 RSA-OAEP + المفتاح الخاص لـ Bob
        ↓
 مفتاح AES (32 بايت) الأصلي ✓
```

الخطوة 2 — فك تشفير الرسالة

```
Ciphertext + مفتاح AES + Nonce
        ↓
   AES-256-GCM Decrypt
        ↓
 "Hello Bob!" ✓
```

الخطوة 3 — التحقق من السلامة

```
الرسالة المستقبَلة
        ↓
   SHA-256 (يحسب من جديد)
        ↓
 a3f8c2d1... (المرسل Hash مقارنة مع الـ)

 ✅ الرسالة أصلية → متطابق
 ❌ تم التلاعب بها → مختلف!
```

الصورة الكاملة

```
ALICE                    NETWORK                  BOB
  |                         |                       |
  |── 1. Gen AES Key ──────►|                       |
  |── 2. AES Encrypt ──────►|                       |
  |── 3. RSA Wrap Key ─────►|                       |
  |── 4. SHA-256 Hash ─────►|                       |
  |                         |                       |
  |      📦 Packet ─────────┼──────────────────────►|
  |                         |                       |── 1. RSA Unwrap Key
  |                         |                       |── 2. AES Decrypt
  |                         |                       |── 3. SHA-256 Verify
  |                         |                       |── ✅ / ❌
```

لماذا هذا التصميم؟

الحل	المشكلة
RSA السريع، ونرسل مفتاحه فقط بـ AES نشفّر بـ	بطيء على النصوص الطويلة RSA
يحلّها بدون الحاجة لقناة آمنة مسبقاً RSA	يحتاج تبادل مفتاح آمن AES
يكتشف أي تعديل ولو حرف واحد SHA-256	لا نعرف إذا عدّل أحد الرسالة